World Models

1. Introducing world models

This chapter covers

- What a world model is and how it differs from LLMs
- Why and where latent-space prediction beats pixel generation
- Breaking down state, dynamics, and planning
- Structuring a minimal predictive agent

When you prompt a language model about something like: “What happens when a cup full of coffee goes off the edge of a table?”, it’ll give you a simple response saying something like: the cup falls, it bounces or breaks depending on the materials involved, and the coffee goes everywhere. That’s correct, but it doesn’t mean the model ‘understands’ the event in the same sense a person does. It doesn’t simulate the scene by tracking the cup when it leaves the table, following the change in its motion, trying to predict where it will land, and choosing whether to catch it or not depending upon the temperature of the coffee.

Let’s consider another example. When a footballer, say Lionel Messi, gets the ball, he scans the field and analyzes all the options he has. Then, while maintaining perfect ball control, he has to decide in an instant whether to pass to teammate A, make a through ball to teammate B making a run, or dribble 5 players by himself, and then pass the ball. There are so many options to choose from. Also, while he’s dribbling, he predicts how some action he takes, like a body feint, would make an opponent react, and uses that prediction as an advantage by going the other way afterwards without pausing and deciding after the defender has reacted. If an AI has to match this level of complexity, it needs a world model.

A world model is inspired by the way we model the world inside our minds and how it works through perception and interaction. It gives the agent a really good idea of what things are likely, probable, or impossible. It tries to predict how its environment will change over time by compressing raw observations into smaller representations, learning how those representations change under the influence of its actions, and using those predictions to analyze all its options by mentally trying them out before physically committing to one.

This predictive power isn't just for robotics though. While we will use the example of a warehouse delivery robot navigating blind corners to ground our concepts throughout this chapter, world models are equally critical in any domain where decisions carry consequences that compound over time. It could be autonomous driving systems predicting pedestrian trajectories, dynamic routing networks preventing traffic jams, or running an autonomous research lab, the underlying requirement would be the same: the agent must imagine multiple futures before acting.

This book isn't just a survey of the existing architectures to build world models, but rather focused on the ideas behind them. You'll keep running into the same three building blocks throughout the book: a state representation, a dynamics model, and a planner. These can be implemented in various ways: diffusion-based generative simulators, latent predictors that never reconstruct a single pixel, recurrent and state-space models built for long sequences, and systems that imagine full rollouts before acting.

The goal is to teach you the intuition behind designing these models, along with the knowledge of architectures. This book assumes the reader has some background or a crash course on PyTorch and how a neural network works. Code is referenced throughout to keep the ideas concrete.

1.1. What is a world model?

A world model is an internal, predictive model of an environment. It keeps a compact representation of the current situation, called the *state*; it learns how that representation changes over time and in response to actions, called the *dynamics*; and it uses both to weigh possible futures before acting, which is *planning*.

World models are based on how humans seem to reason. We don't simply react to something we observe; we carry an internal picture of the world around us, imagine several possible futures, work through the consequences of different actions, and then decide. This human-inspired approach is what makes these models such an interesting area to work in.

There are three important ideas that come to mind. Self-supervised learning helps an AI train itself by learning the inherent patterns in raw and unlabeled data. Temporal modeling is responsible for handling how those internal representations evolve over time. Reinforcement learning uses the resulting predictions to pick the most rewarding actions. Together, these help an agent answer a question: given the current state, what things can happen next, and what actions should it take?

All of this is very useful when a machine has to operate in a dynamic environment rather than just answering questions. Consider a robot that has to interact with an alien object that it hasn't been trained to interact with. If it can imagine what'll happen if it tries to hold it in some particular way, and do that for multiple parallel imagined futures, it's less likely to break it. It's like a routing system that mentally tests several paths and picks the smart one, avoiding a costly mistake without having to backtrack. A language model is enough for most multimodal tasks today, but when decisions carry consequences that play out over time, we need the predictive power of world models.

This concept of predicting the next state of the world depending upon the current state and some action is not new. In fact, it's been around since the 1950s and is called model-predictive control. Modern world models essentially replace those handcrafted control systems with deep learned representations of the environment.

It's a matter of debate whether LLMs, world models, or some other approach will lead to artificial general intelligence. No one knows the answer for sure, but all these models have made us wonder and rethink our definitions of general intelligence, which is for any entity, limited by its abilities of perception, visualization, memory, dimensions, and so on.

Now let's return to our small delivery robot moving towards a blind corner. It knows from its past data that a forklift can cross there every now and then. Its perception consists of a video feed, maybe a depth sensor, or even wheel odometry, all of which is streaming in constantly. That raw data is useless on its own. The world model's first job is to compress all that data into a state, which is a dense vector that has processed usable information about things like where the robot is, what's nearby, and whether anything's moving. Everything redundant, like the exact pixel color of some dot on the floor, is dropped.

This state, alone, is just a snapshot. The second job, and the one that gives a world model its name, is to predict how the state changes, either on its own or in response to some action the robot might try. This is the dynamics model. It's a function of the current state and some candidate action, and outputs the predicted next state. Chaining this function by feeding each predicted state back in as the input for the next step, we generate an imagined rollout. This allows the robot to internally simulate a sequence of future events it hasn't actually experienced yet.

All of this helps in planning. A world model is useful only if its predictions shape the choices made for downstream tasks. It scores all the candidate rollouts against its goal, which here is getting to a destination fast and without hitting anything, and then commits to the first action of the rollout that scores highest overall. It never has to interact and find out in the real world that what it did was bad, with no way to undo it afterward.

Let's walk through this loop from start to finish and track a single decision made by our warehouse robot as it approaches a blind corner.

The process begins in the physical world. The robot's cameras and sensors take in high-resolution video frames of the warehouse(the gray concrete floor, the glare of the overhead lights, and the shadows of the shelves). At this point, the data is just a heavy, noisy stream of raw pixels and sensor readings.

Now, this raw sensory overload flows into the encoder. The encoder acts as a filter, and strips away everything irrelevant like the paint color or the shadows. It compresses only the important stuff, like the fact that the robot is moving at a certain speed, there's a stationary shelf ahead, and a forklift is approaching from the right, into a compact mathematical vector. This small, dense package of information is our state.

The robot needs to decide what to do next. It takes this state and pairs it with several candidate actions, like braking hard, swerving left, or maintaining speed. These pairs are fed into the dynamics model. You can think of the dynamics model as the robot's internal imagination. It acts like a fast-forward button, as it branches the current state into multiple parallel futures and predicts what the next state will look like for each possible action, all without moving an inch in the real world.

Finally, these imagined futures are handed over to the planner. The planner evaluates each rollout against the robot's main goal of reaching the destination quickly without crashing. Braking hard might score high on safety but low on speed. Going left might score terribly if the dynamics model predicts a collision with the shelf. Maintaining speed might result in hitting the forklift.

The planner picks the highest-scoring path, which could be braking hard, and selects only the very first action from that imagined future. It sends this single command back to the physical robot.

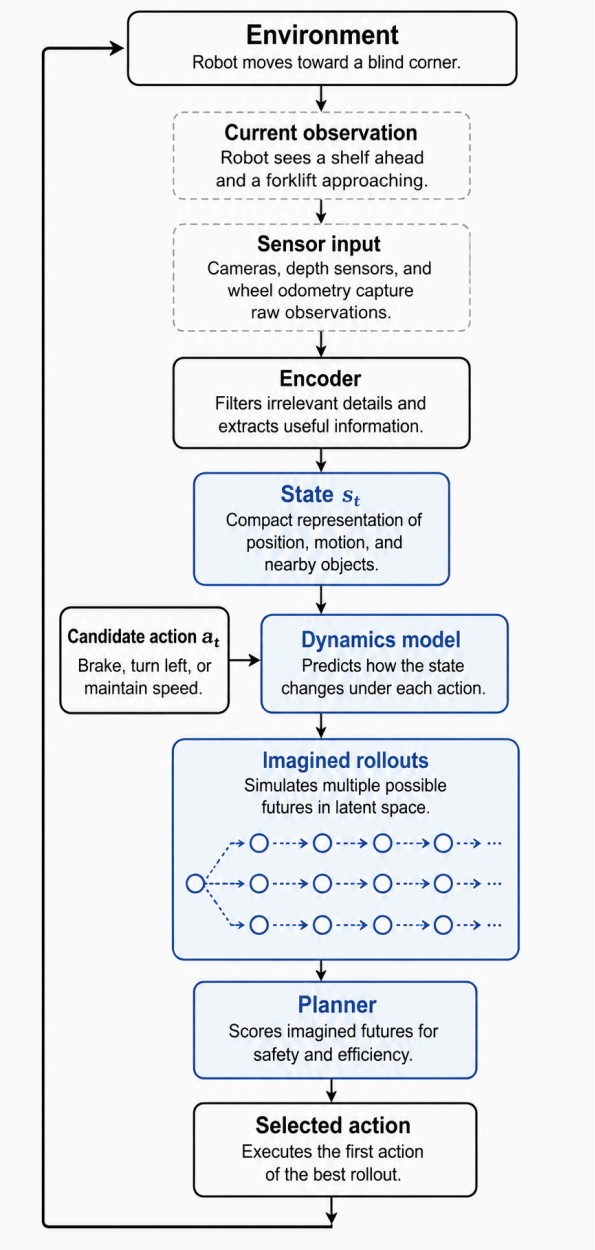


Figure 1.1 The world-model loop applied to our warehouse robot. Raw camera and depth sensor input from the warehouse floor flows into an encoder, which produces a state. The state and a candidate action (like turning left) flow into a dynamics model, which branches into several imagined next states. A planner evaluates each imagined rollout for safety and efficiency, sending the highest-scoring action back to the physical robot. The resulting new observation is fed back into the encoder, and the loop repeats.

The robot hits the brakes in the real world, the cameras capture a brand new observation of the updated environment, and the entire loop starts over.

Different systems handle this loop differently. Some might skip the planner and depend entirely on a deeply trained dynamics model, while others might skip predicting the dynamics altogether. But this specific, continuous cycle of compressing observations, predicting the consequences of actions, and planning a path before physically committing to it, is what people intuitively mean when they say “world model.”

This entire system, however, relies on that very first step: compression. Let’s answer a fundamental question: why compress the state at all instead of just predicting the next raw video frame?

1.2. Prediction versus generation

We’ve been talking about how the state is supposed to be compact. But why? We can just train a model to predict the next raw observation, pixel by pixel, and feeding it the full frame instead of a ‘compressed’ one. Video-generation models do exactly this, and they do it so well that it’s very hard to distinguish real from fake these days.

These generative video models learn the distribution of plausible next frames, and then sample from it to produce something that is indistinguishable from a real world video. Its success is based on how it looks. If it gives a sharp, coherent, and plausible footage a camera could have shot, the model has done its job well. Generation answers the question: “what might this look like?”

An agent acting in the real world asks a different question: “what’s likely to happen if I do this instead of that?” That’s prediction, atleast in the sense that we use it in this book, and the success metric would be how well the decisions are, not on appearance. The warehouse robot doesn’t need to paint multiple frames per second internally, it just needs to know whether maintaining its motion will end up in a collision and slowing down at the corner to look both sides and further decision-making is more optimal. A model doesn’t need to be a great painter in this case, it just needs the sweet spot of important information. Everything beyond that would actually hurt the system by increasing latency, taking more computation and affecting optimal decision-making in tasks downstream. Even if someone has to design an agent which is both: a delivery robot and a video-generator, we would still make two separate architectures.

Moreover, a planner also needs its imagined futures conditioned on candidate actions. It doesn’t ask “what happens next”. It asks “what happens next if I brake, and what happens next if I don’t.” Generative models aren’t designed to answer that question. When we generate an AI video, it shows one likely future, not several parallel ones.

When we work in pixel space, we face three major problems. First, the world is stochastic and it's almost impossible to predict pixel-level future of things like the exact flicker of a bulb, the trajectory of a dust particle flowing in the wind, the motion of leaves on a tree, all of which is just noise. A model scored on pixels can't actually predict any of that. It either averages all the possibilities, which results in a blur, or invents specifics which were never implied. These don't sound that bad, but they actually hurt an agent, because the model's uncertainty about the the wallpaper is expressed in the same currency as its uncertainty about the forklift, which confuses the planner.

Second, the pixel loss doesn't tell which pixels actually matter. The paint color of the warehouse floor doesn't change, making it easily predictable. The problem is that a pixel-based objective rewards getting that paint right as much as tracking the forklift. The training signal cheats the system and only learns to predict the most predictable, even if irrelevant, things rather than whatever matters most for decision-making.

Third is the computation. A single 224x224 color frame has around 150,000 numbers. A compact state would require just a few hundred. If we roll a candidate action sequence twenty steps into its future, and we have fifty such candidates, the planner needs to make a thousand predictions before making a single action, all within a split second, while streaming more raw observations in, and running the loop continuously. So we have a thousand high quality image generations per decision versus a thousand small matrix multiplies. This gap is like the difference between the planning for a datacenter and planning for a robot (figure 1.2).

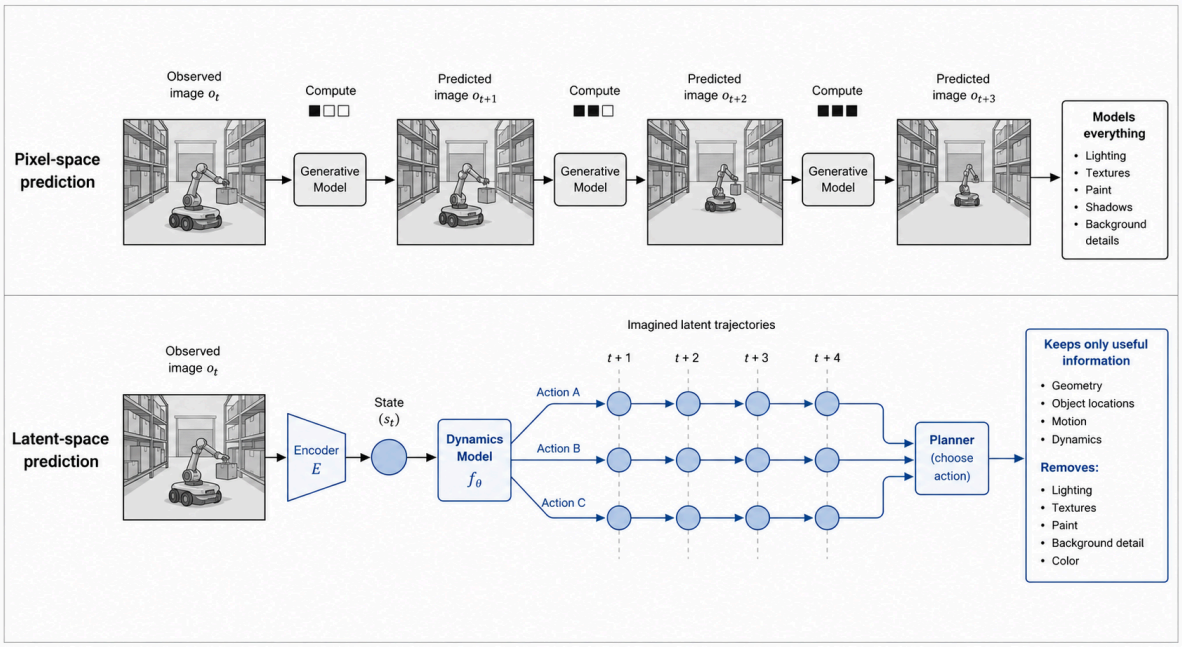


Figure 1.2 Two ways to imagine the future, shown side by side. Top path: pixel-space prediction, where an observed frame feeds a generative model that produces the next frame, which feeds back in to produce the one after, with each step a full image generation and the cost meter climbing at every step. Bottom path: latent prediction, where the same frame passes through an encoder once, and the resulting state rolls forward through a small dynamics model under several candidate actions, branching into multiple cheap imagined trajectories.

To solve all these problems, we need to predict representations rather than complete observations. We have to run the encoder forward in our head rather than running a camera forward in time. Formally, encode the current observation into a state, predict the states that future observations might end up getting compacted to, and train the predictor so its predictions match the representations made by the encoder in future. This way, computation isn't needlessly spent on paint or raindrops. This particular idea of predicting inside a joint embedding space rather than in input space, is the main idea behind Yann LeCun and Meta's JEPA architecture family, which chapter 4 covers deeply. This is also what "latent" means everywhere in this book, whether its latent dynamics or imagined rollout: hidden rather than being directly observed.

This allows the model to be uncertain in the right currency. A pixel predictor is forced to commit to a frame and has to guess some things. A latent predictor can land on a state which might say "something may or may not be about to move through this path, and I can't say much more than that yet," which is a partial description that a single rendered frame cannot express, but is exactly the type of information a good decision usually needs.

It has already been proved that a few hundred numbers with good type and density of relevant information beat pixel predictors at decision-making. Dreamer-style systems, covered in chapter 7, learn dynamics in a latent space and their entire behaviors are trained inside the model's 'dreams', before using that learned behavior in the real world. This only works if the information that the reward function depends on survives into the latent space, which is the task of a well-trained encoder.

However, generation is obviously not the villain of the story, and the "versus" in this section's title is about the perspective. When someone needs to "see" the future, generation is most suitable. The ability to render what a model predicts or dreams, is one of the best debugging tools we have, and a world model that can show its predictions is more reliable than the ones that only output vectors. Generation is even more essential when the system itself is the product, just like in the case of interactive video environments, synthetic training data or playable simulated worlds. Part 4 of this book focuses on building systems like these using diffusion models as their generative engine. But for our agent which has to decide quickly using prediction, generation is an optional, sometimes expensive, sometimes priceless way of showing the result to a human.

With just adding this latent space concept, our loop from section 1.1 survives this whole comparison by compressing the observation, predicting how that compressed state evolves under some candidate actions, and then decide based on those predictions. Let's now understand these three subtasks individually.

1.3. State, dynamics, and planning

The loop from section 1.1 behaves like a story with three parts which have explicit contracts that specify what each takes in, what it hands back, and which design choices stay hidden inside. Most of the real work while building a world model is getting these contracts right, and that's what we discuss in this section. This would help us to make our system modular, to a great extent.

1.3.1. State: the encoder's contract

Everything the agent percepts gets compressed into a dense vector, and that vector must satisfy two conflicting demands. First is sufficiency, which means the state has to preserve relevant and important information from the past that will matter in future. Second is minimality. It has

to keep nothing else, for all the reasons we discussed in section 1.2. A good encoder is the sweet spot between these two demands, and the way we train the encoder determines the negotiation, which is what we will focus on in chapters 3 and 4.

When two separate situations compress down to the same state, we call that aliasing, which is a particular diagnosable way sufficiency can fail. A single camera frame shows you the forklift's location, but says nothing about its velocity. If the state only summarizes the current frame, the two situations: forklift parked at the crossing and forklift passing by the crossing, alias to the same vector. There's nothing much the dynamics model could do apart from predicting one future for two identical yet completely different situations. What it usually does is predict some average future which belongs to neither. The fix is simple, and its written in the contract above: the state shall summarize the history, not the current frame, so that some form of memory lives inside the encoder itself rather than getting bolted on afterward. When this book mentions "state," it means an evolved, compressed record of everything the agent has percepted. It's not just a snapshot.

> ### The Markov property
>
> A system has the Markov property when its future depends on its past only through its present, which means that once you know the current state exactly, the past adds nothing more. Raw camera feeds don't have this property, which is why the encoder has to build a state that can tell you, say, which way the forklift is moving. "Make the state Markov" is shorthand for the sufficiency requirement, and it's the benchmark later chapters use for judging state quality.

Learning the state instead of hand-designing it helps the space itself become meaningful. Two nearby vectors in a well-trained state space aren't just placed there randomly. They represent similar futures, so the robot's path traces a line, which the dynamics model can extend. This doesn't happen magically, it's geometry and optimization at work, and this geometric structure also matters when planning happens inside the space itself rather than just across actions.

1.3.2. Dynamics: the predictor's contract

Current state and a candidate action as input, and a predicted next state as output. This is the learned model that we want to act like a physics engine. It has two jobs at once: it has to simulate how the world evolves on its own, and it has to simulate how the agent's actions

affect that. That second job is what makes it a planning tool and not just a predictor. If we take the action input out, we get a video predictor; if we keep it in, we have a planner that has something to work with.

There are two main problems we face now. First, a learned dynamics model is limited in its reliability to the states and actions it saw during training. Second, a twenty-step imagined rollout means that a one-step model will trust itself 20 times in a row, and it can wander into parts of state space the model never even saw, where its outputs become meaningless and errors compound in such a way that a single one doesn't stand out, so the planner downstream has np way to notice all this. Keeping long rollouts is so hard that chapter 6 is devoted to it, and its discussed in chapter 12 as a failure mode.

This dynamics model slot is filled by sequence models like transformers, recurrent networks, state-space models like Mamba, which are all candidates for the same contract: history and action in, prediction out. This part is also referred to as the memory of the model.

1.3.3. Planning: the decider's contract

Input the current state, the dynamics model, and a scoring function, output is one action. The scoring function determines the actual explicit goals of an agent. "Reach fast and in one piece" has to become numbers attached to the imagined trajectories before scoring starts. Here, we face two main failures of planning: the score lies (reward not specified correctly) or the model lies (predictions are wrong). The first one is a silent bug. The planner is optimizing exactly what we told it to, even if that's not what we actually meant.

The canonical planning algorithm is quite simple actually. Generate some candidate action sequences given the current state, run each one through the dynamics model, score the trajectories that come out, execute the first action of best trajectory, discard rest of the plan, observe the new state, and repeat. That discarding step is the deep part because that's what makes this model-predictive control. Because the model is approximate, and the world is unpredictable, it's not wise to commit to a full trajectory. Just a single action is enough. An imperfect model becomes usable because we have the freedom to replan at every step. This helps in two ways: catching mistakes before they compound to catastrophe and dealing with stochastic things like cats. Chapter 7 focuses on this a lot by building this loop, and even developing its main alternative, which is to compress the search into a policy network that

maps states directly to actions, trained on imagined rollouts, just like Dreamer. In that, you trade runtime compute for training compute, so its faster at decision time, but you lose the freedom to change the reward without retraining.

What we understand from these three contracts is that planning is where computation turns into competence. Given a dynamics model, more rollouts, longer horizon, and a well-defined scoring mechanism, we get better decisions. And planning turns a predictor into an agent. That's what decides what to do with the predictions.

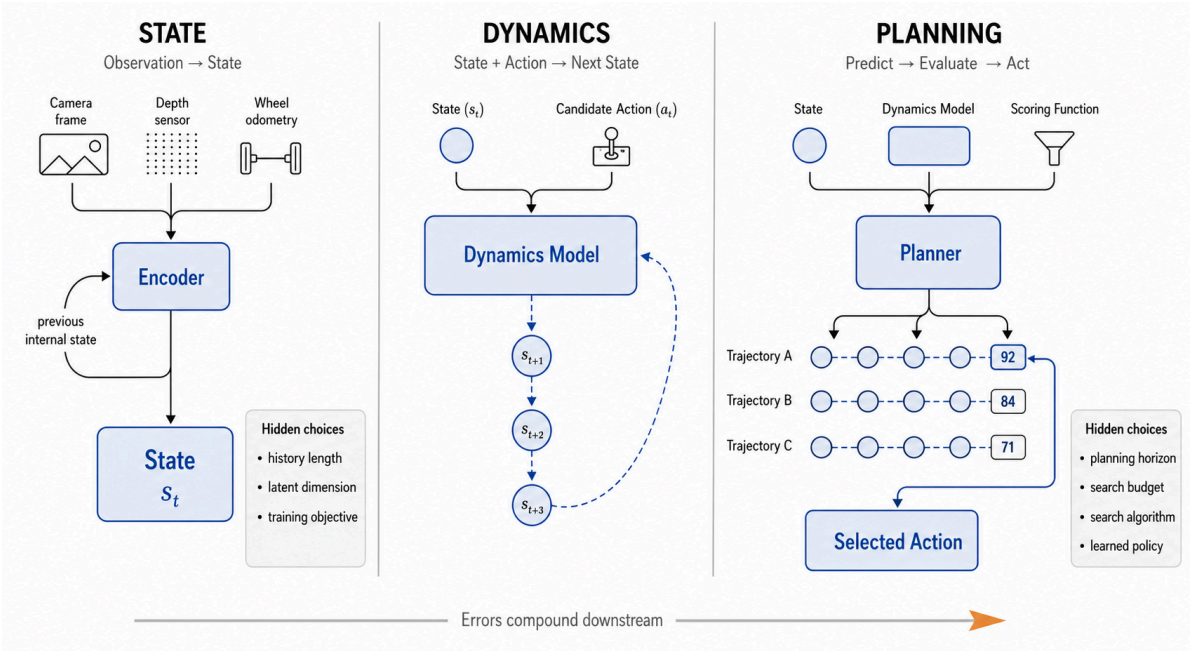


Figure 1.3 The three working parts as software contracts, in three panels.

It's important to note the directions in which errors travel. A bad encoder that aliases two different situations results in the dynamics model getting confused; if the dynamics model drifts, the planner will be confidently wrong. Errors compound downstream. This is is why this book spends its early chapters on states and dynamics, before building a planner. That's the same premise behind this whole section: a state exists, persisting from one tick to next, updated rather than getting rebuilt from scratch every instant. Most AI systems today don't work this way, and that difference turns out to matter more than some architectural choice. Let's discuss that deeply.

1.4. Persistent state in AI systems

Everything we have discussed so far is based on the assumption that something endures over a sequence, from one tick to next. Unusually, this particular method is quite rare in the way AI is practiced today. Consider a transformer model. Mathematically, it's stateless: sequence in, prediction out, internal weights remain untouched. It's just like an HTTP server, "being logged in" is just an illusion maintained by resending the session history with every single request.

Even though today's inference systems fix all this using KV caching and prompt caching, this persistence is a virtue of the serving layer, not the model itself, which remains a pure function with ephemeral memory. All this trickery is just a conversation's paper trail, whereas a world model maintains an active, evolving state of reality that persists even when no one is talking to it.

The world-model loop works like this: state vector stays and just gets updates, not rebuilt, by each new observation. An even better analogy is a running process with its own memory: the heap doesn't recompute itself from source on every instruction, it persists and evolves in place. The warehouse robot doesn't re-derive "the forklift is moving left to right at about walking pace" from raw pixels every frame. It does that once, and then just carries it forward and revises it only when new frames tell it to. This ensures that the state stores conclusions, not observations, and that's what makes memory so cheap.

There's an argument for just having a bigger context window to fix the statelessness and memory problem. But it fails due to three main reasons. First is the cost: Attention's compute grows quadratically with the context length, and even if we do caching, every new token would attends to everything that came before it, making both memory and cost of each step grow rapidly.A fixed-size state would break this relationship as the robot's tenth update would cost just as much as the first one. Secondly, what gets stored? A learned state should gather conclusions, not a transcript like a context window. Also, if the window is finite and episodes aren't, once it fills up, it'll either summarize or just truncate and lose the meaning. A learned state is trained about what to keep and what to discard, and that satisfies the sufficiency requirement we discussed in section 1.3, and the forgetting is intentional. Lastly, we have branching. At every moment, the planner should be able to ask "what if i do A, and what if I do B". When there's no compact object to score and compare, every candidate future in a context-based model is just a separate, expensive pass over an ever-growing transcript, and the futures are merely token sequences. On the other hand, a state vector can be copied fifty ways, rolled forward, and discard by just doing a few small matrix multiplies (figure 1.4).

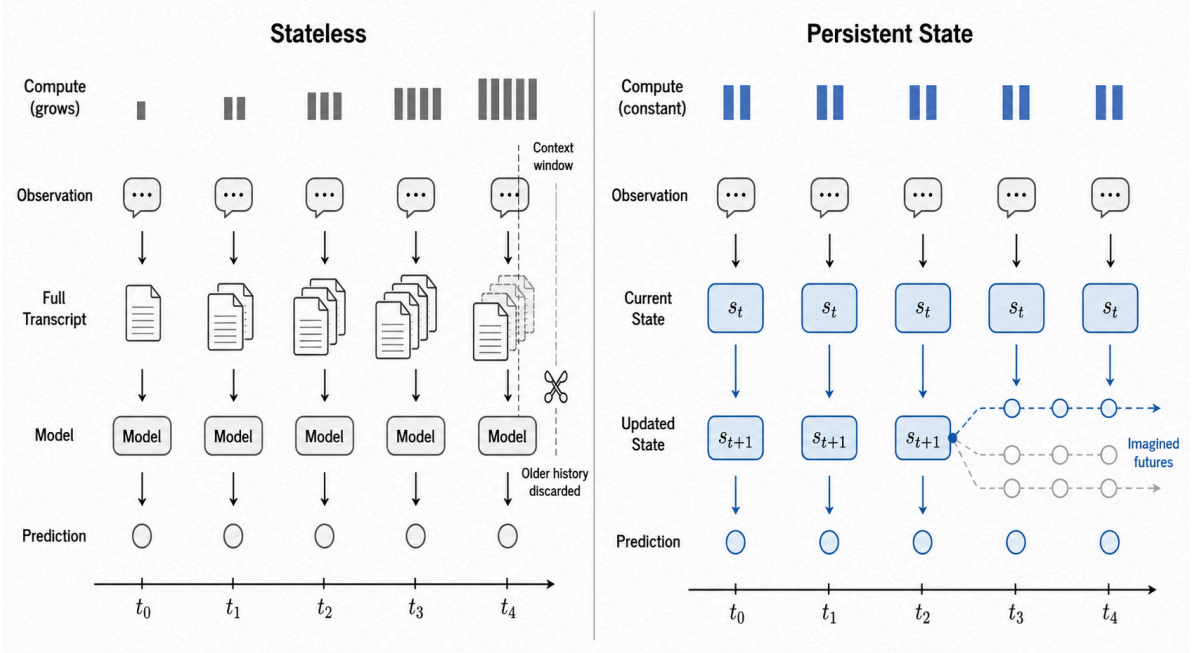


Figure 1.4 Two ways of remembering the past. Left: a stateless system receives the entire conversation history with every request, so both the transcript and the computation grow until the context window is reached and older information must be discarded. Right: a persistent-state system updates a fixed-size state after each observation. The state stays the same size over time, keeps computation constant, and can briefly branch into multiple imagined futures before continuing with a single chosen path.

It's not even a matter of debate anymore whether an agent needs a state, the question is how it gets there. Also, persistence should be a property rather than an architectural workaround. Chapter 5 discusses state-space models like Mamba as a temporal backbone for world models because they're exactly the kind of sequence models we need, the ones that carry a state across steps. This concept is more important than any implementation: an agent operating over time should advance a compact state and evolve it as it goes, not just read bigger transcript every time.

All the pieces from section 1.1 are in place now: a state that persists and evolves, prediction that happens in latent space instead of pixel space, and finally a planner that uses those predictions to make decisions. All of it tries to make an agent perform well in an ever-changing and massively stochastic real world. Now let's discuss why we need all this machinery? What would happen if a system tries to operate without these things.

1.5. Why agents need world models

If we take the world model out of our system, we still have the oldest method of learning, which is to try something, watch what happens, and adjust accordingly. In reinforcement learning, we call this “model-free learning” in which an agent keeps improving its sense of which actions are rewarding in which scenarios, without building any model of how the environment works. This is how most of the well-known game-playing agents were trained in the last decade. But, this requires that the practice must happen in a real setting, one attempt at a time with mistakes and everything.

That’s affordable when the task is just a game because a game instance copies across a datacenter, resets a million times, and runs faster than real time, which helps these agents complete tens of thousands years of practice. But reality doesn’t offer all that. There’s not a collective hive mind with reset, rewind and speedup options. Mistakes are too expensive to be allowed. So these failures, that matter a lot, aren’t even allowed to run.

Usually, we fix that with a simulator, where we try to build a virtual copy of the environment, practice cheaply in that, and then transfer the learning to the real world. But we haven’t solved physics entirely yet. There’s still some seemingly irrelevant things that haven’t been explained or even discovered. Even if we discard all that, it’s really difficult to hand-author faithful simulators of even a single warehouse, if not every warehouse,kitchen, and street on the planet. To solve this sim-to-real gap entirely would be playing god. That’s why we use world models, where we don’t hand-write a simulator, and instead it is learned from the data it percepts, picks up the dynamics of the environment it’s in, forklift habits included, so noone has to guess in advance what was worth modeling.

Once we get a good enough model, the learning becomes rather cheaper. Real sensor data trains the model, which in turn can be used to simulate as many practice sessions as our compute budget allows. The training ground is no longer the real world, which just acts as the raw material now.Everything that reality denied gets inherited with practice. Resets: option to rewind to just before a bad decision in the imagined warehouse and try alternatives. Safety: freedom to crash into a thousand imagined forklifts without any cost. Speed:ability to imagine several futures in parallel, and much faster than reality. And it even allows us something which reality doesn’t offer in principle: counterfactuals. A real world irreversible trajectory can’t answer what would have happened if the agent acted differently in the past. Dreamer-style systems, covered in detail in chapter 7, run this same loop. All the practice is handled by the learned model, and the refined behaviour gets to touch the real world.

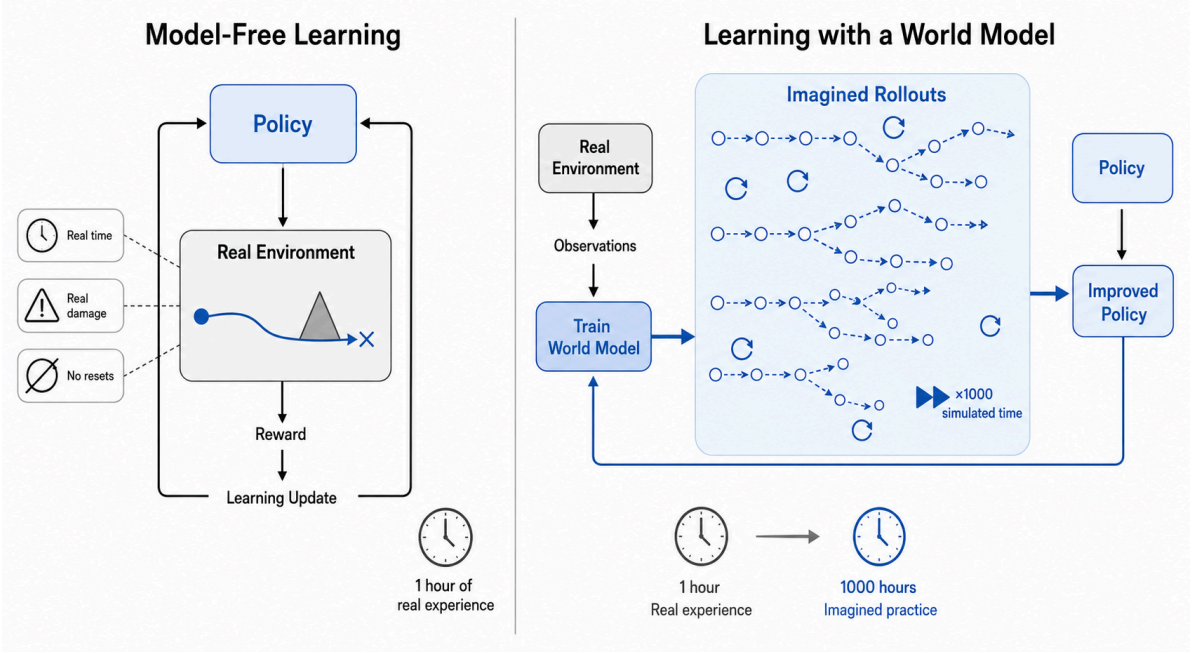


Figure 1.5 Learning without and with a world model, drawn as two loops. Left: the model-free loop, where the agent acts directly in the real environment, receives an occasional reward, and adjusts; annotations mark the three costs: no resets, real damage, real time. Right: the model-based loop, where real experience flows into training the world model; the policy practices inside imagined rollouts that fork, reset, and run faster than real time; only the improved behavior is released into the real environment.

But we know that a model has to be wrong somewhere, and a policy which is trained long enough inside it will eventually find those limitations. If we hard-optimize against a flawed simulator, which is every learned model ever, the policy will just learn to exploit that flaw. It'll find moves that only work in imagination because physics isn't rigid there. There are some ways to defend against such behaviour: keeping the imagined horizons short, tracking the model's uncertainty, refreshing it with real data regularly, and chapters 6 and 12 discuss them in detail. None of that affects the math though. The model that's right most of the time still can turn one hour of reality into thousands of hours of mostly valid practice. But that's the best thing we have, and the atlernative is learning everything the slow, risky, irreversible way in the real world.

Where does all this matter beyond our simple warehouse robot? The same three issues show up everywhere. It could be manipulation or locomotion, where hardware is expensive and the real world isn't that welcoming. It could be driving, where events important to practice, like a child chasing a ball into the street, are rare, unrepeatable, and untestable on a real road, and where every other driver's behavior is to also predict and react to you. It could be routing and logistics, where the reward is to prevent a traffic jam before it even happens by not letting it

build up. Part 3 of this book ends with interactive simulation, where the learned model is our product: environments learned though videos that a human or an agent can actually interact with. So when the mistakes are expensive, resets are impossible, and time doesn't speed up, the agent requires a world model. If none of this is required, like summarizing a document, the simpler tools win.

We've discussed the whys in this chapter. Now let's understand how, in a minimalistic way, by reducing the loop to its individual parts and write a pseudocode for the simplest agent that still qualifies as a world-model system, concrete enough to code quickly, and shaped in a modular way that every later chapter can swap in something more cool in one of its pieces.

1.6. A simple predictive agent

We're ready to move past the abstraction now. A world model, as software, is basically three functions and a loop, and this skeleton is fixed in the entire book. Listing 1.1 shows the entirety of our agent. The encoder, dynamics, and reward function are placeholders, as the strategies to implement those things vary across architectures.

Listing 1.1 A minimal predictive agent (pseudocode)

PSEUDOCODE

```
state = empty_state()                              # nothing known yet
loop at each tick:
    obs   = environment.observe()
    state = encoder.update(state, obs)             # persistent, updated
in place
    best_score, best_plan = -infinity, none
    repeat 50 times:
        plan  = sample_action_sequence(horizon = 20)
        s     = state                              # a copy: imagination
branches here
        score = 0
        for action in plan:
            s     = dynamics.predict(s, action)    # latent rollout,
no pixels
            score = score + reward.estimate(s)
        if score > best_score:
            best_score, best_plan = score, plan
    environment.act(best_plan[0])                  # first action only;
replan next tick
```

Let’s understand this pseudocode slowly. The agent starts with an empty state. It knows nothing at that time, and the first few ticks go towards orientation, as encoder.update processes the observations in. That single line follows from the argument made in section 1.4, which is that state is a variable that lives outside the loop, evolved in place every tick rather than being rebuilt from scratch every time, using full history. When the forklift’s velocity is inferred, it stays in the state until new observations update it.

The middle block is the imagination part. It should be noted that it opens with s = state, which copies the state instead of advancing it. Then, fifty candidate futures branch off from the actual, frozen situation of the robot. The inner for loop is the imagined rollout from section 1.1, in which each candidate is a sequence of twenty actions. predict moves the copied state forward in latent space, one action at a time. The scoring line then implies that reward.estimate reads a state instead of a rendered image. The agent would just know that a rollout ended badly without even decoding it back into pixels, which was the whole point of section 1.2. Now, the reward function is handed to the agent and chapter will show us how the reward and value get learned instead, as heads on top of the state.

Then, to make this an agent instead of just a predictor, we use the line environment.act(best_plan[0]). The planner returns the first action of the best rollout, and the others get thrown away, which is model-predictive control, its purest form, just like we discussed in section 1.3: commit to one action, observe again, replan as per the updated information. By the time the loop comes back around, a new observation has already been compacted and fed into the state. So, if the dynamics model makes a mistake, it usually matters for just one tick.

Now lets discuss what’s fake here. Listing 1.1 learns nothing. The reward is handwritten, the encoder could be just latest sensor readings plus some hand-computed velocity, the dynamics model could all just be a few if-statements. We don’t have any training loop, nothing about uncertainty, no way for anything to improve over time. But none of these gaps require changing this skeleton. Over the next chapters, we’ll swap a trainable piece for each placeholder. In chapters 3 and 4, update will become a learned representation. In chapters 5 and 6, predict becomes a learned latent-dynamics model using state-space. In chapter 7, reward.estimate and the candidate sampler we have here become learned value and policies. In chapter 9 and 10, the dynamics model will grow into a full generative simulator. All these swaps happen in this basic skeleton. That’s why understanding the simple agent here should be treated like a map.

Summary

- A world model is an internal predictive model that represents the current state of an environment, predicts how that state changes, and uses those predictions to guide decisions.
- The state is a compact summary of everything the agent currently needs to know, rather than a copy of every observation it has seen.
- Predicting future states is often much more efficient than repeatedly generating future observations such as images.
- Most world models can be understood as three parts: an encoder that builds the state, a dynamics model that predicts how the state evolves, and a planner that chooses actions from imagined futures.
- Persistent state allows an agent to carry information forward over time without repeatedly processing its entire history.
- A world model makes it possible to imagine many possible futures before acting, allowing mistakes to happen in simulation instead of the real world.
- By moving most learning and planning into imagination, world models reduce the amount of expensive real-world interaction needed to learn effective behavior.

Last updated 2026-08-13 20:38:04 +0530